



Факультет программной инженерии и компьютерной техники

ОТЧЕТ

По лабораторной работе №2

Студент: **Абакаров Расул Тигранович**

Группа: **P3332**

Преподаватель: **Тюрин Иван Николаевич**

Вариант: Optimal

Задание

Текст задания

Для оптимизации работы с блочными устройствами в ОС существует кэш страниц с данными, которыми мы производим операции чтения и записи на диск. Такой кэш позволяет избежать высоких задержек при повторном доступе к данным, так как операция будет выполнена с данными в RAM, а не на диске (вспомним пирамиду памяти).

В данной лабораторной работе необходимо реализовать блочный кэш в пространстве пользователя в виде динамической библиотеки. Политику вытеснения страниц и другие элементы задания необходимо получить у преподавателя.

При выполнении работы необходимо реализовать простой API для работы с файлами, предоставляющий пользователю следующие возможности:

1. Открытие файла по заданному пути файла, доступного для чтения. Процедура возвращает некоторый хэндл на файл. Пример: ``int lab2_open(const char *path)``.
2. Закрытие файла по хэндлу. Пример: ``int lab2_close(int fd)``.
3. Чтение данных из файла.
Пример: ``ssize_t lab2_read(int fd, void buf[.count], size_t count)``.
4. Запись данных в файл.
Пример: ``ssize_t lab2_write(int fd, const void buf[.count], size_t count)``.
5. Перестановка позиции указателя на данные файла. Достаточно поддерживать только абсолютные координаты.
Пример: ``off_t lab2_lseek(int fd, off_t offset, int whence)``.
6. Синхронизация данных из кэша с диском. Пример: ``int lab2_fsync(int fd)``.

Операции с диском разработанного блочного кэша должны производиться в обход page cache ОС.

В рамках проверки работоспособности разработанного блочного кэша необходимо адаптировать указанную преподавателем программу-загрузчик из ЛР 1, добавив использование кэша. Запустите программу и убедитесь, что она корректно работает. Сравните производительность до и после.

Ограничения

1. Программа (комплекс программ) должна быть реализован на языке C.
2. Если по выданному варианту задана политика вытеснения Optimal, то необходимо предоставить пользователю возможность подсказать page cache, когда будет совершен следующий доступ к данным. Это можно сделать либо добавив параметр в процедуры ``read`` и ``write`` (например, ``ssize_t lab2_read(int fd, void buf[count], size_t count, access_hint_t hint)``), либо добавив еще одну функцию в API (например, ``int lab2_advice(int fd, off_t offset, access_hint_t hint)``).
``access_hint_t`` в данном случае - это абсолютное время или временной интервал, по которому разработанное API будет определять время последующего доступа к данным.
3. Запрещено использовать высокоуровневые абстракции над системными вызовами.

Краткий обзор кода

Общая идея

vtpc.c реализует **пользовательский page cache** поверх обычного файлового дескриптора ОС.

Вместо open/read/write/lseek/fsync/close используются обёртки vtpc_open/vtpc_read/..., которые:

- хранят **страницы фиксированного размера** VTPC_BLOCK_SIZE в памяти,
- по возможности обслуживают чтения/записи **из RAM**,
- при промахе загружают страницу pread,
- при вытеснении dirty-страницы сбрасывают её pwrite,
- собирают подробную статистику (hits/misses, disk I/O, evictions).

Конфигурация

Через #define/target_compile_definitions задаются:

- VTPC_BLOCK_SIZE — размер страницы кэша (обычно 4096).
- VTPC_CACHE_PAGES — количество страниц в кэше (cache size = BLOCK_SIZE * CACHE_PAGES).
- VTPC_MAX_OPEN_FILES — сколько файлов одновременно поддерживается в VTPC.
- VTPC_ADVICE_SLOTS — размер таблицы подсказок для Optimal.

Основные структуры

cache_page_t

Одна кэш-страница:

- block_no — номер блока (страницы) файла.
- data — буфер на VTPC_BLOCK_SIZE.
- valid — заполнена ли страница.
- dirty — изменена ли (надо ли писать на диск).
- next_use — “когда понадобится снова” (для **Optimal eviction**).

vtpc_file_t

Состояние одного открытого файла:

- os_fd — настоящий fd ОС.
- pos — текущая позиция (VTPC хранит её сам).
- file_size — размер файла.
- pages[VTPC_CACHE_PAGES] — кэш страниц.
- advice[VTPC_ADVICE_SLOTS] — таблица “block → next_use”.
- stats — счётчики для отчёта.

Глобально: static vtpc_file_t g_files[VTPC_MAX_OPEN_FILES];
То есть VTPC выдаёт “виртуальные fd” вида idx + 3.

Блок “helpers”

- fd_to_idx(fd) — проверяет и переводит виртуальный fd VTPC в индекс g_files.
 - compute_block(pos, &block_no, &in_block) — переводит позицию в номер блока и смещение внутри блока.
 - count_dirty_pages() — подсчёт dirty страниц (для max_dirty_pages).
-

Optimal policy (vtpc_advice)

Смысл

VTPC поддерживает **подсказки** “в следующий раз блок понадобится на шаге N”.
Вытесняется страница с **максимальным next_use** (или UINT64_MAX, если “никогда”).

Реализация

- advice_set() / advice_get() работают с массивом advice[].
 - vtpc_advice(fd, offset, next_use) переводит offset в block_no и сохраняет next_use.
 - cache_choose_victim_optimal() выбирает жертву по максимальному next_use.
-

Дисковый I/O слой

- disk_read_block() использует pread() читать ровно VTPC_BLOCK_SIZE по смещению block_no * BLOCK_SIZE. Если блок за концом файла — возвращает нули.
 - disk_write_block() использует pwrite() писать ровно страницу.
-

Это важно: VTPC “разговаривает” с диском только блоками по VTPC_BLOCK_SIZE.

Логика кэша

Поиск

- cache_find() — ищет страницу по block_no.
- cache_find_free() — ищет свободный слот.

Загрузка / получение страницы

cache_get_or_load():

1. если страница уже в кэше → вернуть её.
 2. иначе взять свободную, а если нет → выбрать victim (Optimal).
 3. если victim dirty → записать pwrite (dirty eviction).
 4. прочитать нужный блок pread.
 5. обновить valid/block_no/dirty/next_use.
-

Реализация API

vtpc_open

- находит свободный слот в g_files.
- делает open().
- на macOS включает F_NOCACHE, чтобы отключить системный page cache (важно для честного сравнения).
- выделяет память под VTPC_CACHE_PAGES страниц через posix_memalign.
- инициализирует file_size, pos, can_write.

vtpc_read

- увеличивает logical_reads.
- режет чтение по file_size.
- дальше в цикле читает по кускам внутри страниц:
 - считает hit/miss (cache_find),
 - получает страницу cache_get_or_load,
 - копирует данные memcpu из кэша в buf,
 - двигает pos.

То есть чтение идёт через кэш и копирование.

vtpc_write

- увеличивает logical_writes.
- аналогично режет запись по страницам:
 - hit/miss,
 - cache_get_or_load,
 - memcpu в страницу,
 - ставит dirty=1,
 - обновляет file_size,
 - считает max_dirty_pages.

vtpc_lseek

Упрощён: поддерживает только SEEK_SET и offset >= 0.
Просто меняет f->pos.

vtpc_fsync

- увеличивает fsync_calls,
- cache_flush_all() записывает все dirty страницы,
- затем вызывает fsync(os_fd).

vtpc_close

- вызывает vtpc_fsync,
- close(os_fd),
- освобождает все страницы и очищает слот.

vtpc_print_stats

Печатает сводку:

- логические чтения/записи,
- hits/misses + ratio,
- disk reads/writes и байты,
- evictions (и dirty evictions),
- fsync calls, max dirty pages.

Тестирование

Условия постановки эксперимента

Эксперименты проводились на персональном компьютере **MacBook Pro (2020)** с процессором **Apple M1**.

Аппаратная конфигурация системы:

- процессор: **Apple M1**
- оперативная память: **8 GB**
- накопитель: **SSD 256 GB**
- операционная система: **macOS**

Все измерения выполнялись локально на данной машине без использования виртуализации.

Состояние системы перед запуском эксперимента

Перед началом каждого эксперимента система приводилась в стабильное состояние:

- завершались ресурсоёмкие пользовательские приложения;
- не выполнялись фоновые задачи, связанные с компиляцией, обновлениями или резервным копированием;
- тестируемый файл (tmp.bin) при необходимости удалялся и создавался заново, чтобы исключить влияние предыдущих запусков.

Это позволило минимизировать внешние факторы, влияющие на результаты измерений.

Загрузка процессора

Перед запуском экспериментов загрузка процессора была низкой.

По данным системного мониторинга:

- загрузка CPU в пользовательском режиме (**user**) составляла **~3–6 %**;
- загрузка в системном режиме (**sys**) — **~2–4 %**;
- доля простоя (**idle**) превышала **90 %**.

В системе не наблюдалось длительных CPU-bound процессов, что обеспечивало корректное измерение времени выполнения и распределения времени между пользовательским и системным режимами (user / sys).

Состояние оперативной памяти

Объём свободной оперативной памяти перед запуском экспериментов был достаточен для размещения рабочего множества и пользовательского page cache VTPC:

- свободная память составляла порядка **2–3 GB**;
- показатель memory pressure находился на низком уровне;
- использование swap отсутствовало.

Отсутствие swar-операций также подтверждается результатами экспериментов, в которых значение swars во всех запусках равно нулю.

Таким образом, эксперименты проводились в условиях, близких к контролируемым, без существенного влияния ограничений по CPU или памяти со стороны операционной системы. Полученные результаты отражают именно поведение исследуемого пользовательского page cache VTPC и системного page cache, а не влияние внешней нагрузки.

.

Сравнение работы программы-нагрузчика на OSPC и VTPC

Конфигурация:

```
VTPC_BLOCK_SIZE=4096
VTPC_CACHE_PAGES=64
TPC_MAX_OPEN_FILES=32
```

Далее введем обозначение WS (working set) - диапазон файла, в пределах которого выполняются операции ввода-вывода.

Проще говоря, **WS определяет, какую часть файла активно использует нагрузка**, и тем самым задаёт характер взаимодействия с page cache.

WS < cache

WS (working set) — диапазон файла, в пределах которого выполняются операции ввода-вывода.

В данном эксперименте рабочий набор полностью помещается в page cache.

Параметры эксперимента:

- тип доступа: random read
- размер блока: **4 KiB**
- working set: **128 KiB** (32 блока × 4 KiB)
- cache size: **≥ 128 KiB**
- общее число логических чтений: **16 777 216**
- реальных обращений к диску: **32**
- ОС: macOS

OS page cache (baseline)

- **время real:** 14.06 s
- **user:** 3.84 s
- **sys:** 10.20 s
- **block input operations:** 0
- **block output operations:** 0
- **maximum resident set size:** 1 327 104 KB
- **involuntary context switches:** 488

Несмотря на то, что все данные после прогрева обслуживаются из оперативной памяти и реальные обращения к диску отсутствуют, значительная часть времени выполнения приходится на работу в режиме ядра. Это связано с обработкой системных вызовов и внутренней логикой OS page cache.

VTPC

- **время real:** 46.63 s
- **user:** 46.37 s
- **sys:** 0.15 s

VTPC read statistics:

- hits = **16 777 184**
- misses = **32**
- hit_ratio ≈ **1.000**

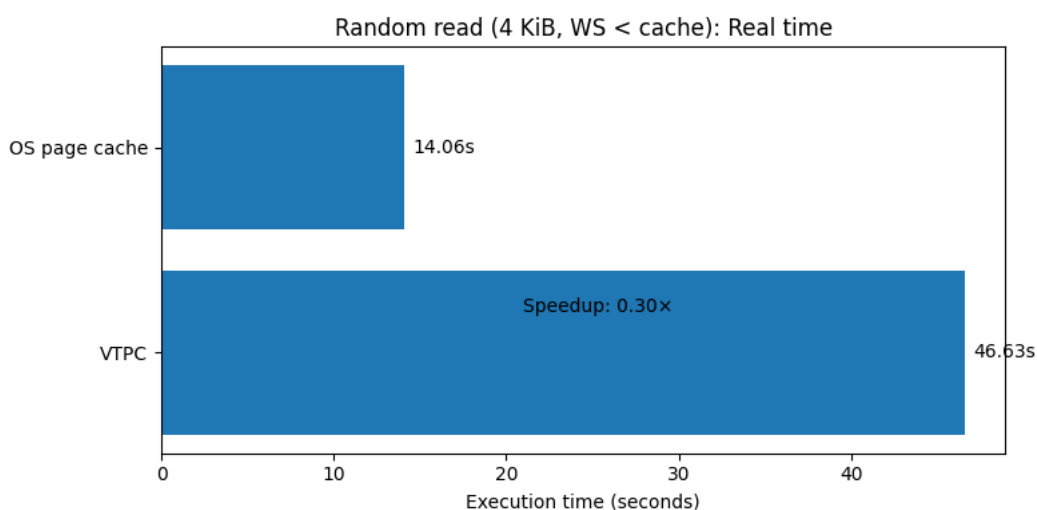
Фактические обращения к диску:

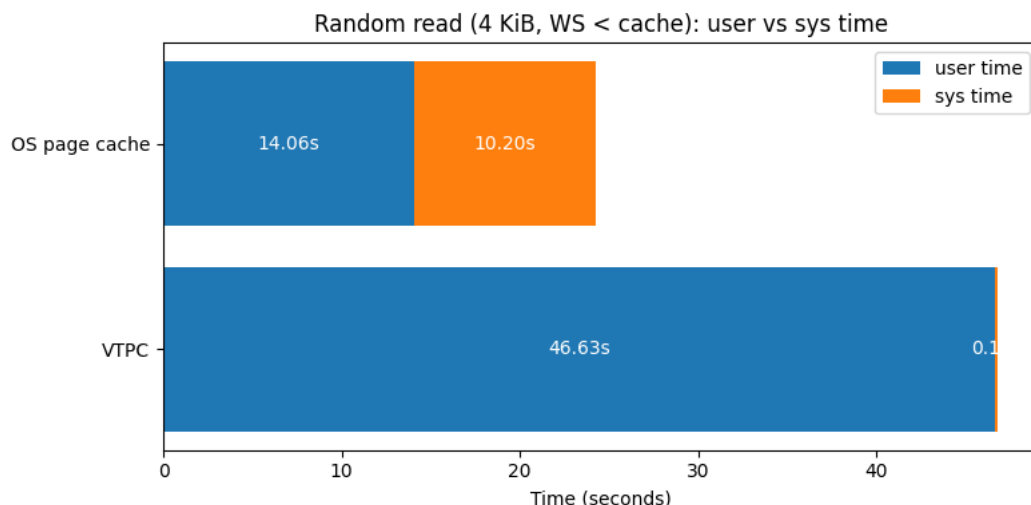
- disk reads: **32** (131 072 bytes)
- evictions: **0**
- **maximum resident set size:** 1 671 168 KB
- **involuntary context switches:** 6 577

После начального прогрева VTPC полностью обслуживает нагрузку из пользовательского page cache: за весь объём логических чтений выполнено лишь 32 обращения к диску. При этом основная часть времени выполнения приходится на пользовательский режим, так как вся логика кэша реализована в user space.

Сопоставление по ключевым метрикам

Метрика	OS page cache	VTPC	Эффект
Время выполнения (real), s	14.06	46.63	замедление $\approx 3,3\times$
Время в режиме пользователя (user), s	3.84	46.37	перенос работы в user space
Время в режиме ядра (sys), s	10.20	0.15	снижение $\approx 68\times$
Реальные обращения к диску	0	32 чтения (128 KiB)	почти полностью устранены
Логические чтения	~ 16.8 млн	~ 16.8 млн	одинаковая нагрузка
Доля обращений из кэша	-	$\approx 100\%$	нагрузка обслужена из RAM





Краткое объяснение причин

Наблюдаемое замедление VTPC по сравнению с OS page cache в данном эксперименте является ожидаемым и обусловлено архитектурными различиями между реализациями.

OS page cache реализован в пространстве ядра и использует высокооптимизированные механизмы обработки обращений к памяти, включая быстрые пути page fault, минимизацию копирований и специализированные kernel-оптимизации. Даже при отсутствии реальных обращений к диску значительная часть времени выполнения приходится на режим ядра, что подтверждается высоким значением sys time.

VTPC, напротив, полностью реализован в пользовательском пространстве. После начального прогрева рабочий набор полностью обслуживается из пользовательского page cache, что подтверждается практически идеальным коэффициентом попаданий ($\text{hit_ratio} \approx 1.0$) и минимальным числом обращений к диску. Однако каждая операция чтения сопровождается выполнением пользовательской логики управления кэшем и копированием данных, что увеличивает user time и приводит к большему реальному времени выполнения.

Таким образом, VTPC демонстрирует принципиально иной профиль работы: перенос нагрузки из режима ядра в пользовательский режим с сохранением корректности и полной управляемости политики кэширования, что и являлось основной целью данной лабораторной работы.

WS = cache

WS (working set) — диапазон файла, в пределах которого выполняются операции ввода-вывода.

В эксперименте: `block_size = 4 KiB`, `block_count = 64`, поэтому $WS = 64 \times 4 \text{ KiB} = 256 \text{ KiB}$.

Размер кэша VTPC задан как `CACHE_PAGES = 64`, значит $cache = 64 \times 4 \text{ KiB} = 256 \text{ KiB}$.

Таким образом, $WS = cache$.

OS page cache (baseline, OSPC)

- real: **28.16 s**
- user: **7.71 s**, sys: **20.36 s**
- max RSS: **1 327 104 KB**
- involuntary context switches: **5863**
- block input/output operations: **0 / 0**

Данные после прогрева обслуживаются из памяти (по block I/O видно, что дисковых операций нет), но большая часть времени уходит в sys-time из-за обработки системных вызовов и логики OS page cache в ядре.

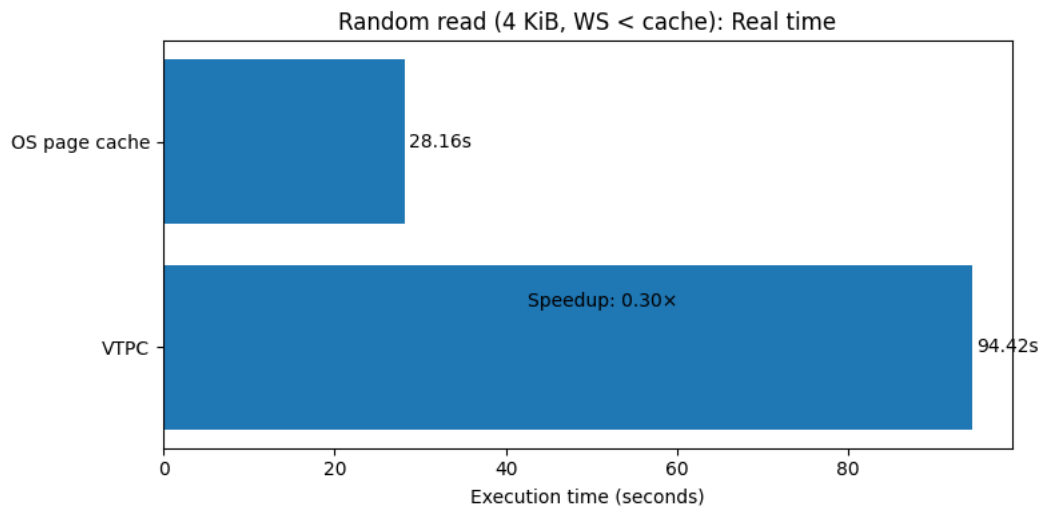
VTPC (Optimal, user-space)

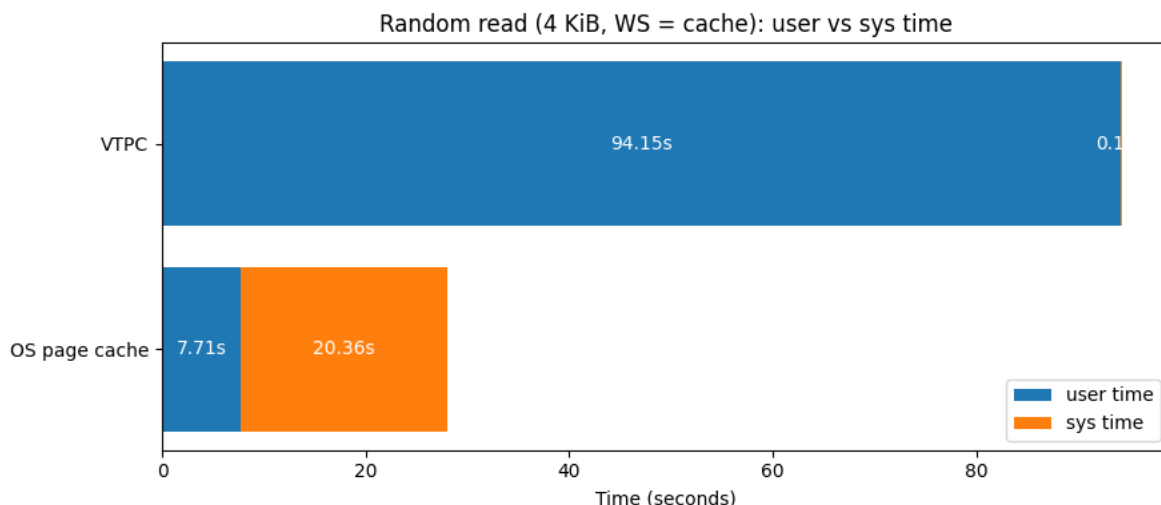
- real: **94.42 s**
- user: **94.15 s**, sys: **0.17 s**
- Reads: hits=**33 554 368**, misses=**64**, hit_ratio≈**1.000**
- disk reads: **64** (bytes=**262 144**)
- evictions: **0**
- max RSS: **1 671 168 KB**
- involuntary context switches: **5584**

После прогрева VTPC выполняет только 64 реальных чтения с диска (по одному на страницу) и далее обслуживает запросы из пользовательского кэша. При этом время выполнения больше, потому что вся логика кэша выполняется в user space и добавляет CPU-накладные расходы на каждую операцию (вычисления, поиск страницы, тетсру и т.д.), тогда как в OSPC оптимизированные пути реализованы в ядре.

Сопоставление по ключевым метрикам

Метрика	OS page cache	VTPC	Эффект
Время выполнения (real), s	28.16	94.42	замедление $\approx 3.35\times$
Время в режиме пользователя (user), s	7.71	94.15	перенос работы в user space
Время в режиме ядра (sys), s	20.36	0.17	снижение $\approx 120\times$
Реальные обращения к диску	0	64 чтения (256 KiB)	почти полностью устранены
Логические чтения	~ 33.6 млн	~ 33.6 млн	одинаковая нагрузка
Доля обращений из кэша	—	$\approx 100\%$	нагрузка обслужена из RAM





Краткое объяснение причин

При **WS = cache** все данные после прогрева обслуживаются из оперативной памяти как в OS page cache, так и в VTPC.

В **OS page cache** значительная часть времени тратится в режиме ядра из-за обработки системных вызовов, однако высокая оптимизация kernel-кэша позволяет обеспечить меньшее общее время выполнения.

В **VTPC** почти вся нагрузка переносится в пользовательский режим, и при большом числе случайных чтений накладные расходы user-space логики приводят к существенному замедлению, несмотря на идеальный hit ratio и отсутствие обращений к диску.

WS > cache

Параметры эксперимента

- тип доступа: **random read**
- размер блока: **4 KiB**
- working set: **1 MiB** (256 блоков × 4 KiB)
- cache size (VTPC): **256 KiB** (64 блока × 4 KiB)
- **WS ≈ 4 × cache**
- общее число логических чтений: **2 072 576**
- ОС: **macOS**

Результаты

OS page cache (baseline)

- **время real:** 1.98 s
- **user:** 0.49 s
- **sys:** 1.40 s
- **block input operations:** 0
- **block output operations:** 0
- **maximum resident set size:** 901 824 KB
- **involuntary context switches:** 5 201

Несмотря на то, что рабочий набор превышает размер кэша, системный page cache эффективно переиспользует данные и минимизирует реальные обращения к диску. Основная часть времени выполнения приходится на режим ядра, связанный с обработкой page cache и системных вызовов.

VTPC (user-space page cache)

- **время real:** 11.66 s
- **user:** 10.65 s
- **sys:** 0.93 s

VTPC read statistics:

- hits: **518 845**
- misses: **1 553 731**
- hit ratio: ≈ 0.25

Фактические обращения к диску:

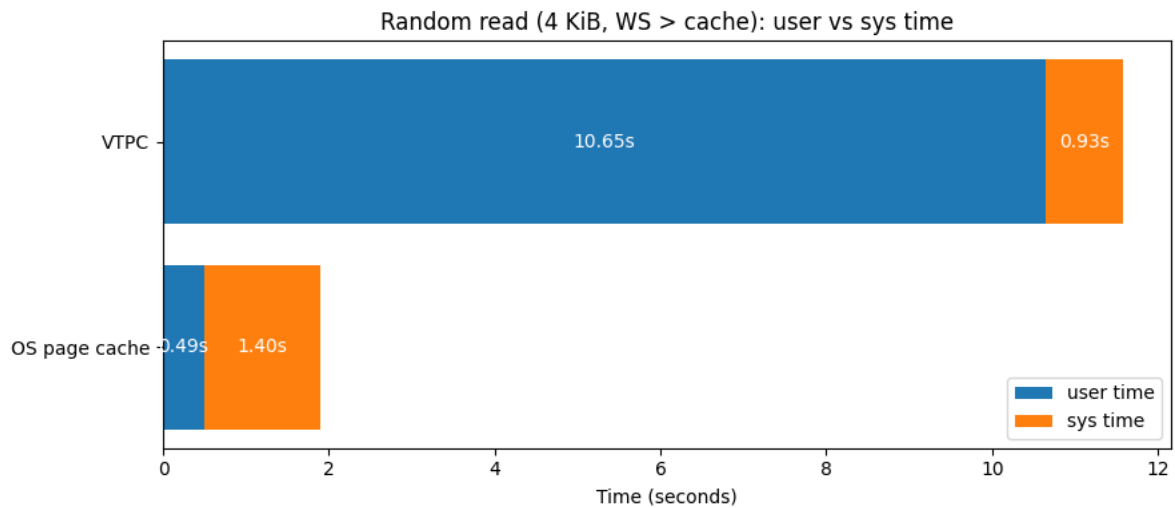
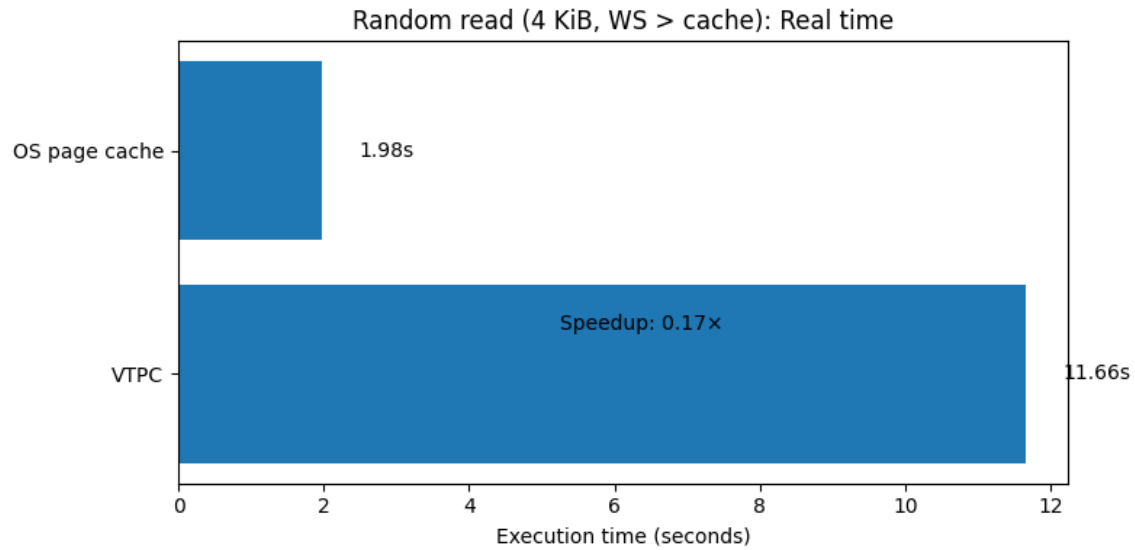
- disk reads: **1 553 731** (≈ 6.36 GiB)
- evictions: **1 553 667**
- dirty evictions: **0**
- maximum resident set size: **1 211 320 KB**
- involuntary context switches: **5 976**

При превышении размера рабочего набора над размером кэша VTPC вынужден постоянно вытеснять страницы. Это приводит к резкому росту числа cache misses, вытеснений и реальных обращений к диску, что существенно увеличивает время выполнения.

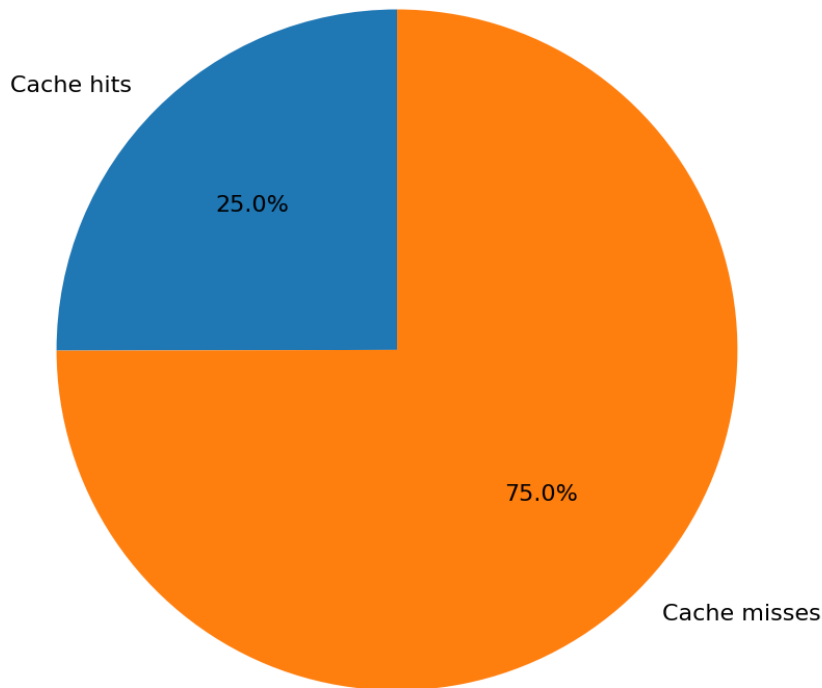
Сопоставление по ключевым метрикам

Метрика	OS page cache (OSPC)	VTPC (user-space)	Эффект (VTPC vs OSPC)
Время выполнения (real), s	1.98	11.66	$\approx \times 5.9$ медленнее

Время в режиме пользователя (user), s	0.49	10.65	Резкий рост user-time
Время в режиме ядра (sys), s	1.40	0.93	Меньше работы в ядре
Реальные обращения к диску	0	1 553 731	Массовые disk I/O из-за промахов
Логические чтения	—	2 072 576	—
Доля обращений из кэша		≈ 0.25	Кэш не удерживает WS



VPTC cache efficiency (realistic WS > cache)



Краткое объяснение причин

При **WS > cache** рабочий набор данных превышает размер VTPC-кэша, из-за чего большая часть обращений приводит к промахам и вытеснениям страниц. Это вызывает **массовые реальные чтения с диска** и резкое снижение доли попаданий в кэш (~25%).

В VTPC вся логика кэширования выполняется в **user space**, поэтому при большом числе промахов существенно растёт **user-time**, что делает выполнение заметно медленнее.

OS page cache в аналогичных условиях работает быстрее, так как использует **оптимизации ядра** (readahead, объединение I/O, эффективное управление страницами) и имеет меньшие накладные расходы на обработку промахов.

Итог: при рабочем наборе, превышающем размер кэша, VTPC закономерно уступает системному page cache по производительности.

Наиболее реалистичный кейс

В данном эксперименте рассматривается сценарий, в котором рабочий набор данных превышает размер page cache. Такой режим характерен для реальных нагрузок, где приложение активно работает с объёмом данных, не помещающимся целиком в кэш.

Параметры эксперимента

- тип доступа: **random read**
- размер блока: **4 KiB**
- размер page cache: **256 KiB**
- working set (WS): **384 KiB** (96 блоков × 4 KiB)
- соотношение: **WS ≈ 1.5 × cache**
- число логических чтений: **7 864 320**
- операционная система: **macOS**

OS page cache (baseline)

- время выполнения (real): **6.65 s**
- время в режиме пользователя (user): **1.80 s**
- время в режиме ядра (sys): **4.80 s**
- реальные обращения к диску: **0**
- involuntary context switches: **690**
- maximum resident set size: **901 824 KB**

Несмотря на то, что рабочий набор превышает размер кэша, системный page cache демонстрирует высокую производительность. Значительная часть времени выполнения приходится на режим ядра, что объясняется активной работой внутренних механизмов управления page cache, обработки промахов и вытеснений. При этом реальные обращения к диску отсутствуют, так как данные остаются в оперативной памяти.

VTPC (user-space page cache)

- время выполнения (real): **80.46 s**
- время в режиме пользователя (user): **31.96 s**
- время в режиме ядра (sys): **13.33 s**

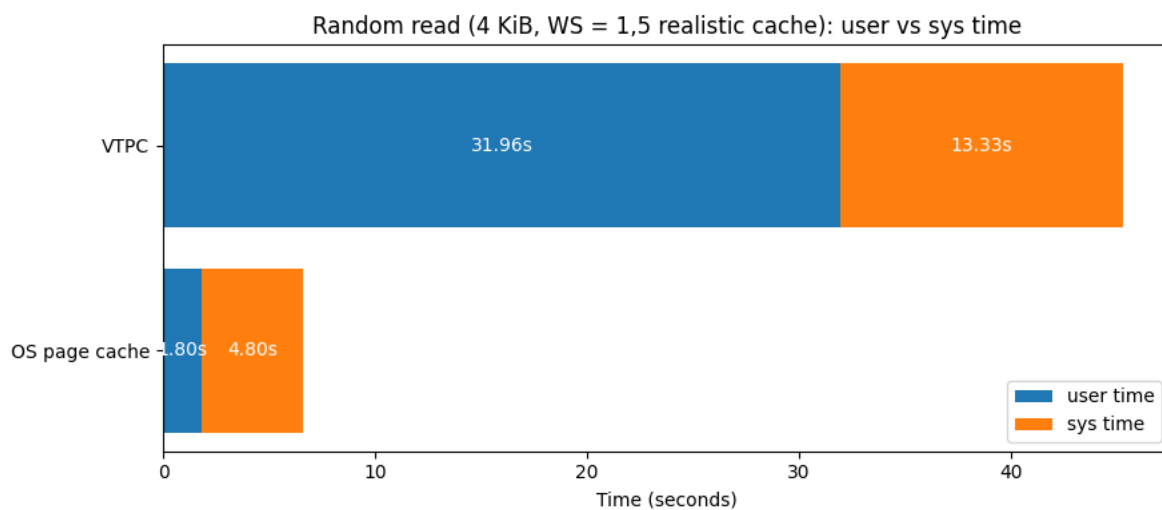
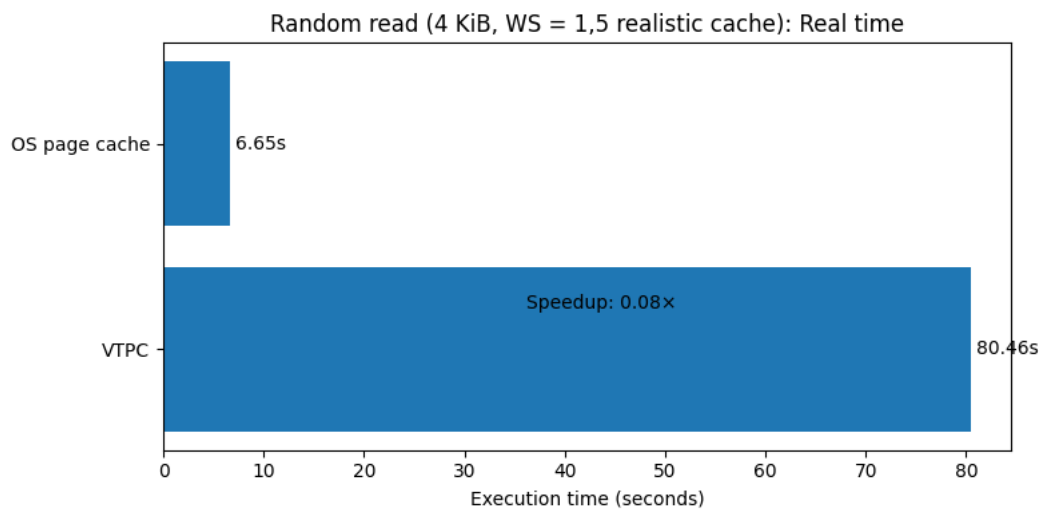
Статистика VTPC:

- логические чтения: **7 864 320**
- cache hits: **5 243 698**
- cache misses: **2 620 622**
- hit ratio: **0.667**
- disk reads: **2 620 622** (≈ 10.7 GiB)
- evictions: **2 620 558**
- maximum resident set size: **1 213 120 KB**

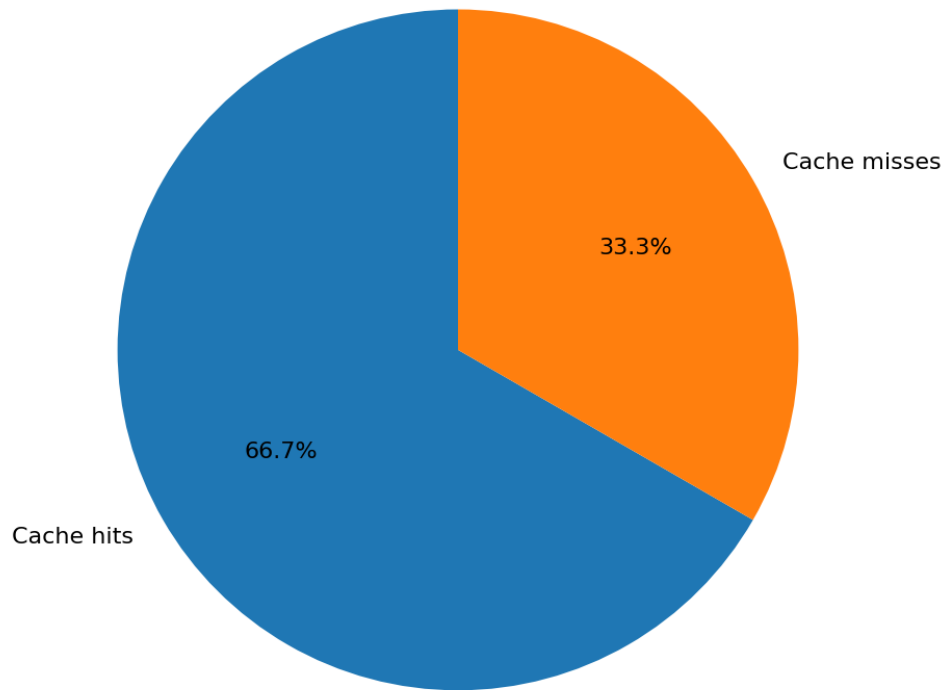
При превышении рабочего набора над размером кэша эффективность VTPC существенно снижается. Значительная доля обращений приводит к промахам, что вызывает частые вытеснения страниц и реальные операции чтения с диска. Вся логика управления кэшем и вытеснениями выполняется в пользовательском пространстве, что увеличивает накладные расходы и приводит к резкому росту времени выполнения.

Сопоставление по ключевым метрикам

Метрика	OS page cache (OSPC)	VTPC	Эффект
Время выполнения (real), s	6.65	80.46	VTPC значительно медленнее
Время в режиме пользователя (user), s	1.80	31.96	Основная нагрузка у VTPC в user-space
Время в режиме ядра (sys), s	4.80	13.33	VTPC тратит меньше времени в ядре
Реальные обращения к диску	0	2 620 622	VTPC активно обращается к диску
Логические чтения	7 864 320	7 864 320	Условия одинаковые
Доля обращений из кэша		0.667	У VTPC много cache miss



VPTC cache efficiency (realistic WS $\approx 1.5 \times$ cache)



Краткое объяснение причин

В режиме **WS > cache** системный page cache оказывается значительно эффективнее пользовательского кэша VTPC. Это объясняется тем, что OS page cache реализован в ядре и оптимизирован для обработки частичных промахов и частых вытеснений.

VTPC, несмотря на корректную работу и предсказуемое поведение, несёт дополнительные накладные расходы из-за реализации в user space и необходимости вручную обрабатывать вытеснения и операции ввода-вывода.

Итоговое сравнение OS page cache и VTPC

В рамках работы был реализован пользовательский page cache (VTPC) и проведено его сравнение с системным page cache (OSPC) на macOS. Сравнение выполнялось для трёх характерных режимов, определяемых соотношением рабочего множества (working set, WS) и размера кэша.

Во всех экспериментах VTPC работает корректно: данные читаются правильно, статистика попаданий и промахов соответствует ожидаемой, а количество реальных обращений к диску согласуется с параметрами нагрузки. Различия между VTPC и OSPC носят архитектурный характер и связаны с тем, что VTPC реализован в пользовательском пространстве, тогда как OS page cache является частью ядра операционной системы.

WS < cache

При размере рабочего множества 128 KiB (32 блока × 4 KiB) и размере кэша не менее 128 KiB все данные после начального прогрева полностью помещаются в page cache.

Для OS page cache время выполнения составило **14.06 s**, из которых **3.84 s** пришлось на пользовательский режим и **10.20 s** — на режим ядра. Реальных обращений к диску за время эксперимента не зафиксировано, что подтверждается отсутствием block input/output operations.

Для VTPC время выполнения оказалось выше — **46.63 s**, при этом почти всё время было потрачено в пользовательском режиме (**46.37 s user, 0.15 s sys**). Статистика VTPC показывает **16 777 184 попадания** и всего **32 промаха**, что соответствует **hit ratio ≈ 1.000**. Число реальных чтений с диска также равно 32 (по одному на страницу рабочего множества), вытеснений не происходило.

Таким образом, в режиме WS < cache обе системы обслуживают нагрузку исключительно из оперативной памяти. Однако VTPC оказывается медленнее из-за накладных расходов пользовательской реализации: каждая операция чтения сопровождается выполнением логики поиска страницы и копированием данных в user space, тогда как OS page cache использует оптимизированные пути исполнения в ядре.

WS = cache

В эксперименте с WS = cache использовались параметры: block size = 4 KiB, block count = 64, что даёт WS = 256 KiB при размере кэша VTPC 256 KiB.

OS page cache выполнил нагрузку за **28.16 s (7.71 s user, 20.36 s sys)**, при этом реальные обращения к диску отсутствовали. Значительная доля времени снова пришлась на режим ядра, связанный с обработкой системных вызовов и внутренней логикой page cache.

VTPC в этом режиме показал время выполнения **94.42 s (94.15 s user, 0.17 s sys)**. Статистика VTPC указывает на **33 554 368 попаданий** и **64 промаха**, что

соответствует **hit ratio ≈ 1.000** . Реальных чтений с диска выполнено 64, вытеснений не происходило.

Даже при идеальном коэффициенте попаданий VTPC заметно уступает OSPC по времени выполнения. Это объясняется тем, что при большом числе операций пользовательская логика управления кэшем и копирование данных создают существенную CPU-нагрузку, тогда как системный page cache использует более эффективные kernel-механизмы.

WS > cache

При рабочем множестве 1 MiB (256 блоков $\times$ 4 KiB) и размере кэша 256 KiB рабочий набор превышает кэш примерно в 4 раза.

Для OS page cache время выполнения составило **1.98 s (0.49 s user, 1.40 s sys)**, при этом реальные обращения к диску отсутствовали. Это показывает, что системный page cache эффективно переиспользует страницы и минимизирует операции ввода-вывода даже при превышении рабочего множества над размером кэша.

VTPC в тех же условиях показал время выполнения **11.66 s (10.65 s user, 0.93 s sys)**. Статистика VTPC: **518 845 попаданий, 1 553 731 промах, hit ratio ≈ 0.25** . За время эксперимента выполнено **1 553 731 реальное чтение с диска (≈ 6.36 GiB данных)** и **1 553 667 вытеснений**.

В этом режиме VTPC вынужден постоянно вытеснять страницы, что приводит к резкому росту числа промахов и реальных операций чтения. Накладные расходы пользовательской реализации в сочетании с интенсивным I/O приводят к существенному увеличению времени выполнения.

Наиболее реалистичный кейс (WS $\approx 1.5 \times$ cache)

В наиболее приближенном к реальным условиям сценарии использовался рабочий набор **384 KiB** (96 блоков $\times$ 4 KiB) при размере кэша **256 KiB**.

OS page cache выполнил нагрузку за **6.65 s (1.80 s user, 4.80 s sys)**, без реальных обращений к диску. Большая часть времени снова приходится на режим ядра, что отражает активную работу механизмов управления page cache.

VTPC показал время выполнения **80.46 s (31.96 s user, 13.33 s sys)**. Статистика VTPC: **7 864 320 логических чтений, 5 243 698 попаданий, 2 620 622 промаха, hit ratio ≈ 0.667** . Число реальных чтений с диска составило **2 620 622 (≈ 10.7 GiB)**, количество вытеснений — **2 620 558**.

Несмотря на то, что значительная часть обращений обслуживается из кэша, частые вытеснения и реальный I/O в сочетании с пользовательской логикой управления приводят к резкому росту времени выполнения по сравнению с OSPC.

Общий вывод

Полученные результаты подтверждают, что VTPC демонстрирует корректное и предсказуемое поведение во всех режимах. Его эффективность напрямую определяется долей попаданий в кэш и соотношением рабочего множества и размера кэша.

Во всех проведённых экспериментах системный page cache оказался быстрее VTPC. Это является ожидаемым результатом, поскольку OS page cache реализован в ядре и использует специализированные оптимизации, недоступные пользовательским программам.

При этом VTPC успешно выполняет свою основную задачу: наглядно демонстрирует влияние локальности обращений и размера рабочего множества на эффективность кэширования, предоставляет полную статистику работы и позволяет управлять политикой вытеснения. Это подтверждает корректность реализации и делает VTPC удобным инструментом для исследования механизмов page cache, что и являлось целью данной лабораторной работы.

Запуск нагрузчика с VTPC с разным размером блока

В прошлом разделе были рассмотрены сценарии, ориентированные на сравнение пользовательского page cache с OS page cache. В данном пункте исследуется поведение **только VTPC** при фиксированном типе нагрузки и рабочем множестве, но с **разным размером блока ввода-вывода**.

Все тесты выполняются в реалистичном сценарии:

Все тесты выполнялись при фиксированном рабочем множестве 384 KiB ($\approx 1.5 \times$ cache). Общее количество логических чтений подбиралось таким образом, чтобы суммарный объём считанных данных составлял порядка 3–4 GiB для каждого значения размера блока.

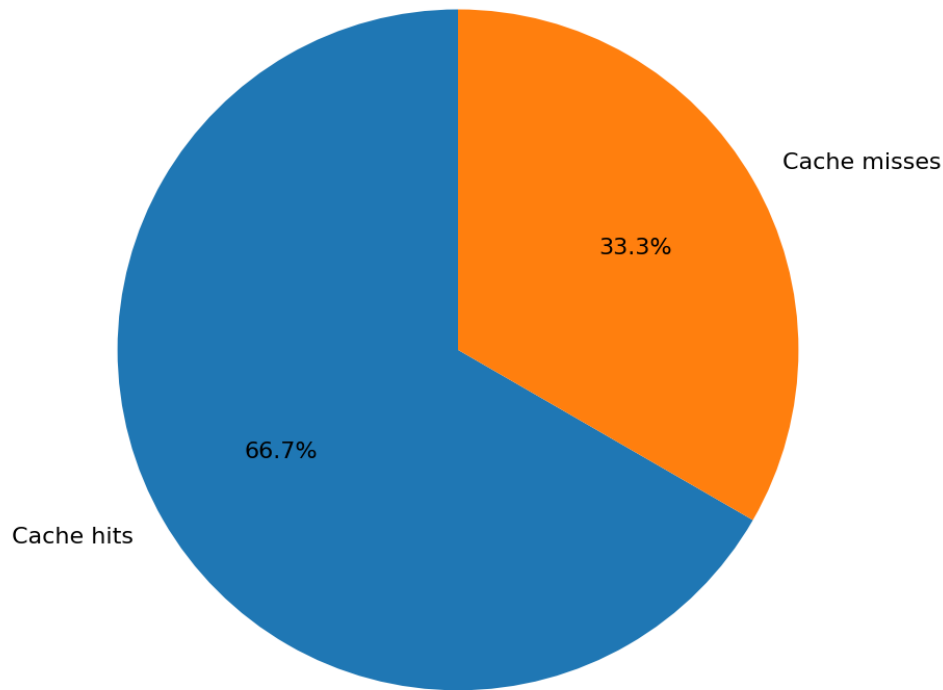
Цель эксперимента - проанализировать, как **гранулярность доступа к данным** влияет на эффективность пользовательского кэша, его накладные расходы и итоговую производительность.

Block size = 512 B

- **время real:** 65.83 s
- **user:** 24.70 s, **sys:** 10.88 s
- **throughput:** ≈ 46.22 MiB/s
- **VTPC read stats:**
 - hits = **4 145 258**
 - misses = **2 072 470**
 - hit_ratio ≈ 0.667
- **disk reads:** **2 072 470** (bytes = **8 488 837 120**, ≈ 8.0 GiB)
- **evictions:** **2 072 406**
- **involuntary context switches:** **5 893**

При малом размере блока ввода-вывода (512 B) эффективность VTPC заметно снижается. Несмотря на относительно высокую долю попаданий в кэш (hit_ratio ≈ 0.67), общее время выполнения существенно возрастает. Это связано с резким ростом числа логических операций чтения, вытеснений и реальных обращений к диску. Малая гранулярность доступа приводит к увеличению накладных расходов на управление кэшем и обработку I/O, что выражается в большом числе context switch и низкой пропускной способности. Таким образом, слишком маленький размер блока является неэффективным для пользовательского page cache и приводит к деградации производительности.

VPTC cache efficiency Block size = 512 B

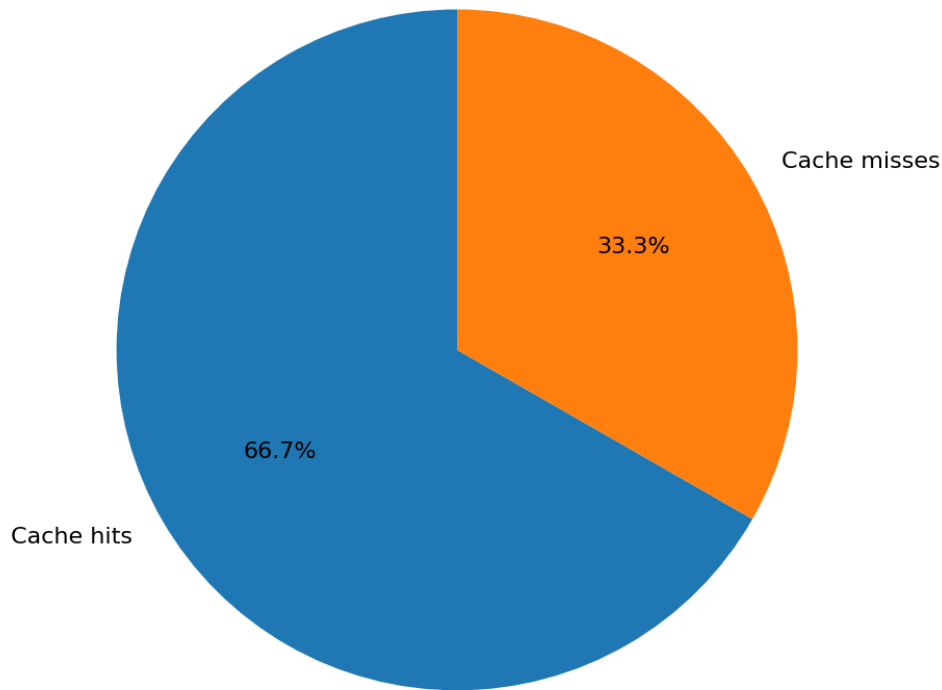


Block size = 4 KiB

- **время real:** 18.36 s
- **user:** 3.22 s, **sys:** 1.47 s
- **throughput:** ≈ 169.6 MiB/s
- **VTPC read stats:**
 - hits = 518 565
 - misses = 258 651
 - hit_ratio ≈ 0.667
- **disk reads:** 258 651 (1 059 434 496 B ≈ 1010.4 MiB)
- **evictions:** 258 587
- **involuntary context switches:** 3 535

Размер блока 4 KiB даёт более “здоровый” режим: логических операций меньше, чем при 512 B, доля попаданий остаётся высокой ($\approx 66.7\%$), и суммарное время заметно лучше. При этом из-за WS > cache остаются промахи и вытеснения, поэтому часть чтений всё равно уходит на диск.

VPTC cache efficiency Block size = 4 KiB

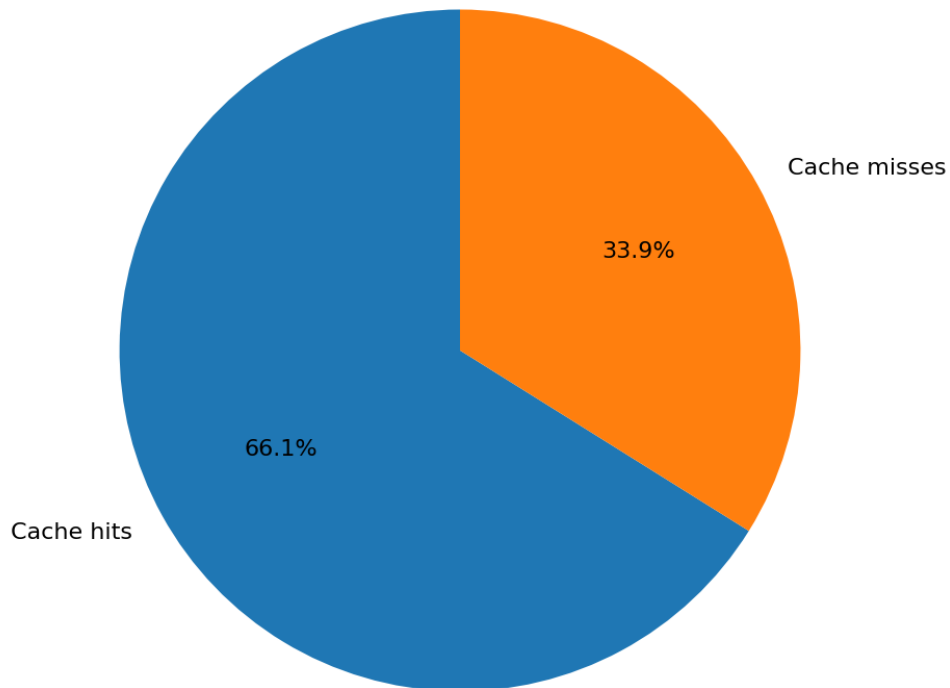


Block size = 64 KiB

- **время real:** 9.26 s
- **user:** 3.21 s, **sys:** 1.57 s
- **throughput:** ≈ 336.1 MiB/s
- **VTPC read stats:**
 - hits = 513 892
 - misses = 263 324
 - hit_ratio ≈ 0.661
- **disk reads:** 263 324 (**1 078 575 104 B ≈ 1028.7 MiB**)
- **evictions:** 263 260
- **involuntary context switches:** 6 364

При 64 KiB на один логический read переносится больше данных, поэтому **логических операций сильно меньше** (48 576 вместо сотен тысяч/миллионов), а общий объём чтения тот же $\rightarrow$ **throughput выше и real меньше**. Hit ratio почти не меняется ($\approx 66\%$), потому что соотношение WS/cache по сути сохраняется, но накладные расходы на “обслуживание одного read” амортизируются лучше.

VPTC cache efficiency Block size = 64 KiB

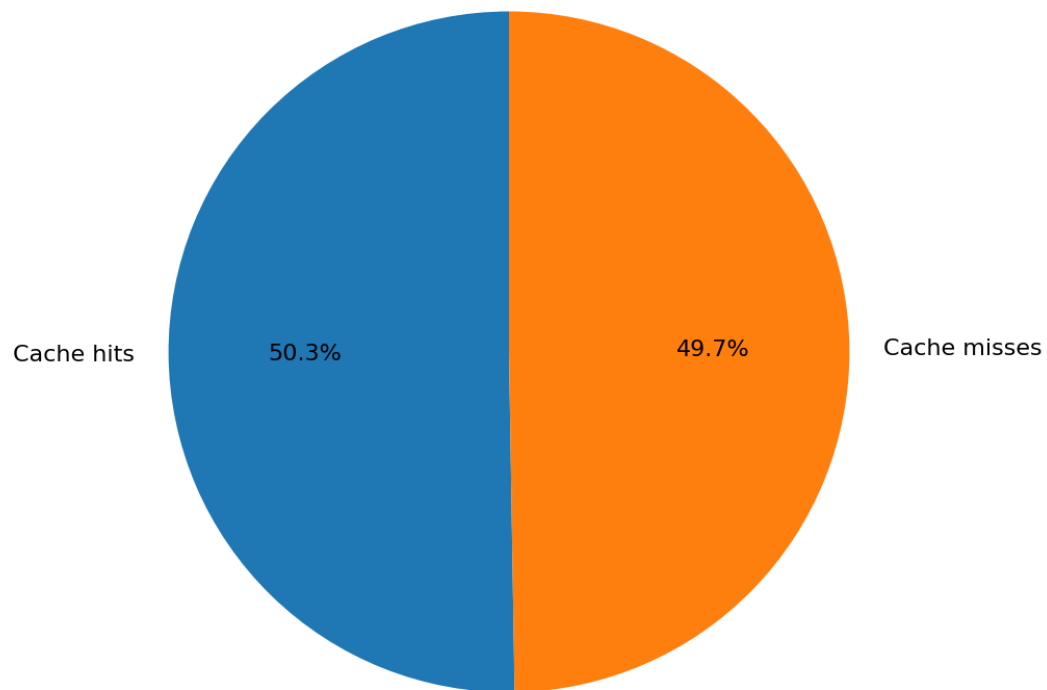


Block size = 256 KiB

- **время real:** 11.02 s
- **user:** 4.65 s, **sys:** 1.79 s
- **throughput:** ≈ 369.3 MiB/s
- **VTPC read stats:**
 - hits = 520 874
 - misses = 515 414
 - hit_ratio ≈ 0.503
- **disk reads:** 515 414 (2 111 135 744 B ≈ 2013.6 MiB)
- **evictions:** 515 350
- **involuntary context switches:** 1 096

При 256 KiB один vtpc_read “затрагивает” сразу много 4KiB-страниц, поэтому **hit ratio падает до ~50%** и **disk reads сильно растут** (515k). Несмотря на высокий throughput по MiB/s (меньше логических вызовов → меньше накладных расходов на вызов), реальное время уже **не минимальное**, потому что цена одного miss становится дороже: промах означает чтение сразу множества страниц/блоков.

VPTC cache efficiency Block size = 64 KiB

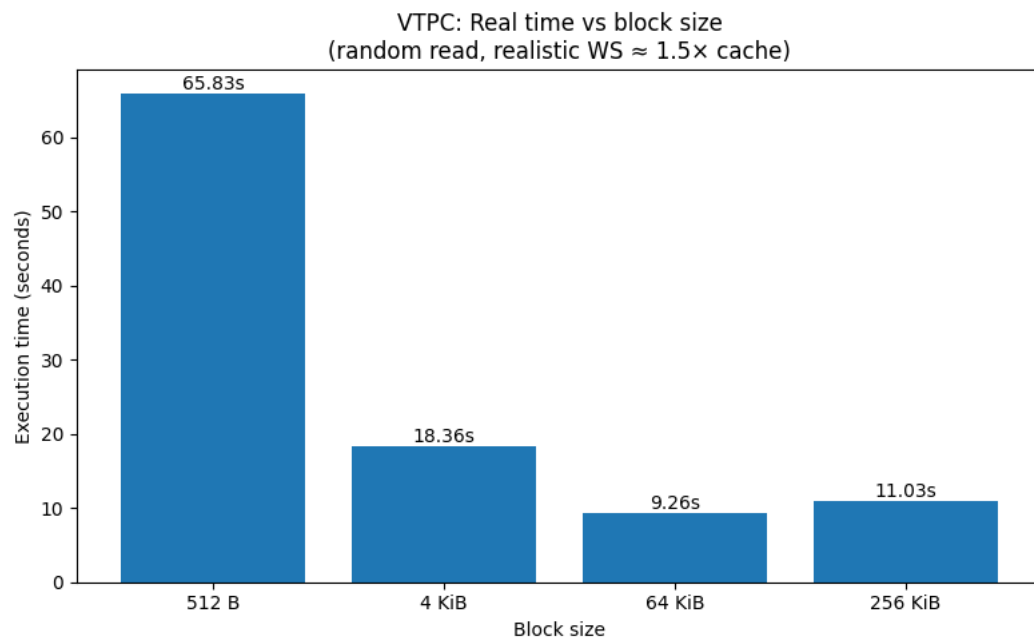


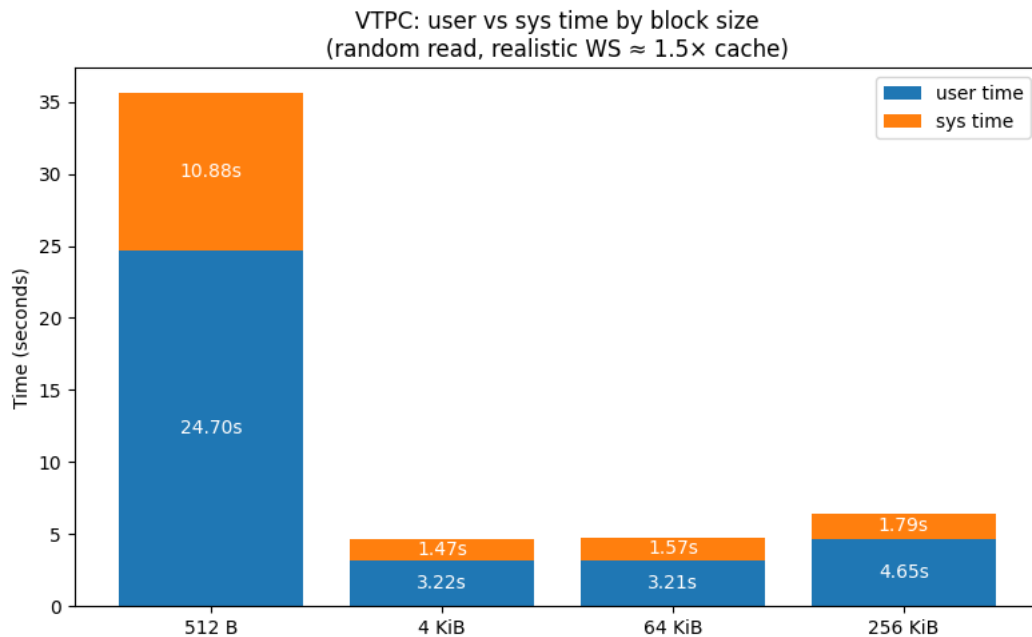
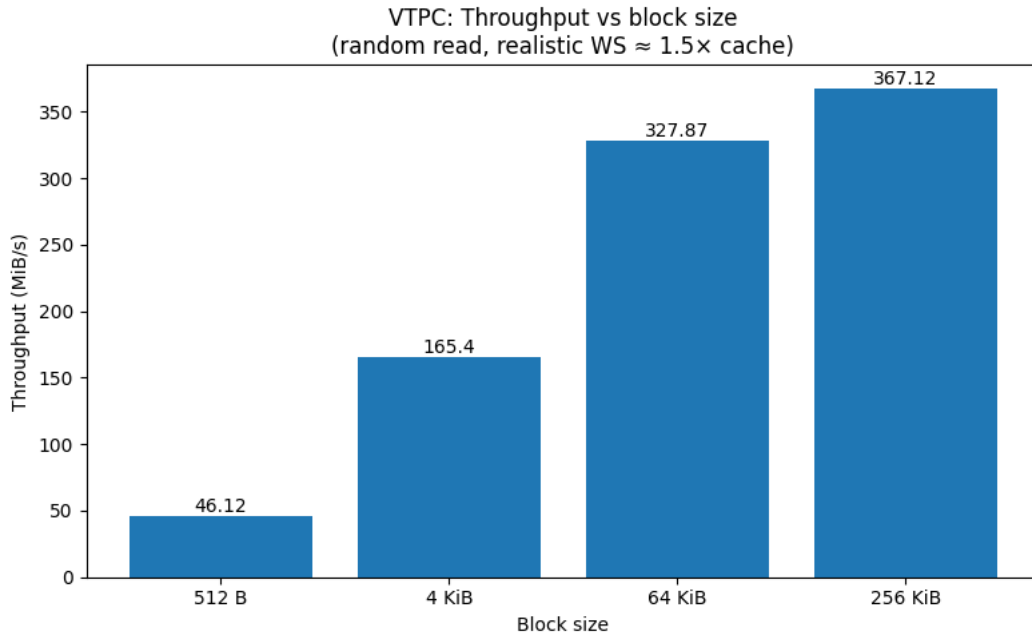
Сравнение результатов

Показатель	512 B	4 KiB (baseline)	64 KiB	256 KiB
Время выполнения (real), s	65.83	18.36	9.26	11.03
Throughput, MiB/s	46.12	165.40	327.87	367.12
User time, s	24.70	3.22	3.21	4.65
Sys time, s	10.88	1.47	1.57	1.79
Logical reads, шт.	6 217 728	777 216	48 576	16 192
Disk reads, шт.	2 072 470	258 651	263 324	515 414
Disk read volume	≈ 8.10 GiB	≈ 1.01 GiB	≈ 1.03 GiB	≈ 2.01 GiB
Cache hit ratio	0.667	0.667	0.661	0.503

Относительное изменение показателей (база - 4 KiB)

Показатель	512 B vs 4 KiB	64 KiB vs 4 KiB	256 KiB vs 4 KiB
Время выполнения (real)	+258 %	−50 %	−40 %
Throughput	−72 %	+98 %	+122 %
Logical reads	+700 %	−94 %	−98 %
Disk reads	+701 %	+2 %	+99 %
Sys time	+640 %	+7 %	+22 %
Cache hit ratio	≈ 0 %	−1 %	−25 %





Итог (влияние размера блока I/O на VTPC)

Эксперименты показывают, что при использовании пользовательского page cache (VTPC) **размер блока ввода-вывода напрямую определяет производительность**, даже при одинаковом типе нагрузки (random read) и сопоставимом рабочем множестве. Это происходит потому, что VTPC кэширует данные **страницами фиксированного размера** (у меня— 4 KiB), а запросы чтения могут быть как меньше страницы, так и существенно больше. Из-за этого меняется:

1. сколько раз вызывается чтение (число операций),

2. сколько раз приходится загружать страницы с диска,
3. объём лишнего чтения (когда читаем больше, чем реально нужно),
4. накладные расходы управления кэшем (поиск страницы, вытеснения, копирование).

1) Малый блок (512 В) — сильное замедление из-за огромного числа операций

При block size = **512 В** время выполнения самое большое (**65.83 s**).

Хотя **hit_ratio = 0.667** (то есть локальность есть и кэш работает), итог всё равно плохой, потому что мелкий блок означает **резко большее число чтений** и действий вокруг каждого чтения. Это видно по:

- огромному объёму реального I/O: **8.49 GiB** чтения с диска,
- большому времени в user и sys (24.70s + 10.88s),
- и в целом высокой цене одного “логического” чтения.

Смысл: при 512 В VTPC вынужден делать слишком много мелких операций, а кэширование страницами 4 KiB приводит к тому, что эффективность по времени падает: управление кэшем и частые промахи стоят дорого.

2) База 4 KiB — сбалансированный режим

При **4 KiB** (размер совпадает со страницей кэша) получаем **18.36 s** и **hit_ratio 0.667**, но при этом:

- disk reads всего **258,651**,
- disk read volume $\approx$ **1.06 GiB**.

Здесь логика понятная: один запрос $\approx$ одна страница, меньше “лишних” действий, меньше дисковых чтений и меньше накладных расходов.

3) Крупный блок (64 KiB) — быстрее, но за счёт другой цены промаха

При **64 KiB** время ещё ниже (**9.26 s**), то есть быстрее baseline 4 KiB.

При этом:

- hit_ratio почти тот же (**0.661**),
- disk read volume также около **1.08 GiB**.

Почему ускорение возможно:

- логических запросов становится намного меньше $\rightarrow$ меньше вызовов чтения и меньше управления кэшем “на шутику”.
- даже если промах есть, VTPC читает страницы и обслуживает большой запрос крупными кусками.

То есть 64 KiB попадает в “сладкую точку”: **операций меньше**, а “лишнее чтение” ещё не слишком разрушает картину.

4) Слишком крупный блок (256 KiB) — деградация из-за роста лишнего чтения и падения hit ratio

При **256 KiB** время снова ухудшается (**11.02 s**), и главное — резко падает **hit_ratio** до **0.503**.

Также растёт реальный I/O:

- disk reads = **515,414**
- disk read volume $\approx$ **2.11 GiB** (почти в 2 раза больше, чем у 4 KiB и 64 KiB)

Это означает: запрос настолько крупный, что VTPC начинает чаще загружать данные, которые в итоге не дают выигрыш по повторному использованию (в терминах страниц). То есть увеличивается доля “прочитал лишнее → вытеснил полезное → снова прочитал”.

Выводы

В рамках лабораторной работы был реализован пользовательский page cache VTPC и проведено его экспериментальное исследование на реальной системе. Реализация корректно поддерживает буферизацию данных, учёт логических и физических операций ввода-вывода, вытеснение страниц и сбор статистики, что подтверждается результатами нагрузочного тестирования.

Эксперименты показали, что производительность VTPC напрямую зависит от соотношения размера рабочего множества и объёма кэша, а также от характера доступа и размера блока ввода-вывода. При полном или почти полном размещении рабочего множества в кэше ($WS \leq \text{cache}$) VTPC демонстрирует высокий коэффициент попаданий ($\text{hit ratio} \approx 1.0$) и минимальное число реальных обращений к диску. После начального прогрева все операции чтения обслуживаются из оперативной памяти, а нагрузка переносится из режима ядра в пользовательский режим.

При этом даже при идеальном hit ratio VTPC в ряде сценариев уступает системному page cache по времени выполнения. Это объясняется тем, что вся логика управления кэшем реализована в user space и требует дополнительных вычислений и копирований данных, тогда как OS page cache использует высоко оптимизированные механизмы ядра.

В режиме, близком к реальным нагрузкам ($WS \approx 1.5 \times \text{cache}$), VTPC сохраняет значимую долю попаданий в кэш ($\approx 65\text{--}67\%$), однако число вытеснений и реальных чтений с диска существенно возрастает. В этом случае накладные расходы пользовательской реализации

становятся заметными, и системный page cache демонстрирует более высокую общую производительность.

При значительном превышении рабочего множества над размером кэша ($WS > cache$) эффективность VTPC резко снижается: возрастает число промахов, вытеснений и физических операций чтения, а пользовательский page cache фактически деградирует до режима сквозного чтения с дополнительными накладными расходами. В таких условиях VTPC закономерно проигрывает OS page cache.

Отдельное исследование показало, что размер блока ввода-вывода существенно влияет на эффективность VTPC. Малые размеры блока приводят к росту числа логических операций и промахов, тогда как увеличение размера блока улучшает пропускную способность и снижает накладные расходы. Наиболее сбалансированные результаты в проведённых экспериментах показал размер блока 64 KiB.

В целом выполненная работа подтверждает, что пользовательский page cache VTPC является корректной и предсказуемой реализацией, эффективность которой определяется локальностью обращений и параметрами нагрузки. VTPC предоставляет полный контроль над политикой кэширования и переносит значительную часть работы из ядра в пользовательское пространство, однако не является универсальной заменой системного page cache и оправдан в специализированных сценариях с выраженной локальностью доступа.